



Cinvestav



COCYTEN
CONSEJO DE CIENCIA Y TECNOLOGÍA
DEL ESTADO DE NAYARIT

LAB 3 Adaptative LUT

Student: Alonso Emmanuel López Macías

Email's: usuario_cocytén_32_x@cinvestav.mx
alemlopezma@gmail.com

Student: Cesar Eduardo Inda Cenicerós

Email's: usuario_cocytén_32_x@cinvestav.mx
cesar.inda@iteso.mx

Full Professor: Dr. Martín González Pérez

10/04/2025

CINVESTAV, Av Del Bosque 1145, El Bajío 45017, Zapopan Jalisco.

COCYTEN, Boulevard Luis Donaldo Colosio. Ciudad Industrial, 63173 Tepic Nayarit.

Technical Report

® CINVESTAV - COCYTEN

Abstract: *FPGA Terasic*

Board – ALTERA Cyclone IV EP4CE115F29C7N

IDE – Quartus Prime 18.1

Boolean Algebra

Digital Gates & Logical Comparators

Git & GitHub Control Version

Keywords: *Parity Bit, Combinational Logic, Even Parity, Odd Parity & Bit Word.*

Table of Contents

Table of Contents.....	ii
Table of Figures	iii
1.- INTRODUCTION	0
1.1.- Requirements.....	0
2.- FUNCTIONAL DESCRIPTION	1
2.1.- True Tables	3
2.2.- Results.....	3
2.3.- GitHub Repository	5
3.- CHALLENGUES/ISSUES.....	5
4.- CONCLUSIONS.....	6

Table of Figures

Figure: 1 LUT_Table	0
Figure: 2 Equation_LUT_8Bit	0
Figure: 3 Megafunction_Mux	1
Figure: 4 Mux_Properties.....	1
Figure: 5 AdaptativeLUT_Circuit.....	2
Figure: 6 RTL_AdaptativeLUT	2
Figure: 7 Pin_Planner.....	2
Figure: 8 TrueTables_LUT.....	3
Figure: 9 TestCombinations.....	3
Figure: 10 Test1	4
Figure: 11 Test2	4
Figure: 12 Test3	4
Figure: 13 Test4	4
Figure: 14 GitHub Repository	5

1.- INTRODUCTION

In this practice, we created an adaptive LUT, taking into account the internal multiplexing functionality of digital systems. This was done with the intention of understanding that an adaptive LUT allows us to integrate this multiplexing through different bit words, while also using various selectors to perform different internal combinations using digital logic gates. It's important to understand that physically, a LUT resembles a multiplexer (MUX), but one of the key differences is that the MUX is used to route data, whereas the LUT is used to evaluate logic functions. With the help of **"mega functions"** available in the Quartus tool, we can also modify the number of data inputs, which in turn automatically changes the selector inputs.

1.1.- Requirements

- Design an adaptive LUT capable of selecting between 8 values.
- Implement the circuit on the FPGA.
- Use switches SW[0] to SW[7] for input data.
- Use switches SW[15] to SW[17] as the selector.
- Use LED LEDR[0] for the output bit.

The composition of the truth table consists of 128 combinations, defined by the base element where "n" = "number of selectors" and "2^n" = "number of input bits."

n	2 ⁿ
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128

Figure: 1 LUT_Table

The equation that determines the behavior of the LUT is as follows:

EQUATION FOR LUT 8 BIT & 3 SELECTORS BY $n \rightarrow 2^n$

$$Y = A_0 \overline{S_0} \overline{S_1} \overline{S_2} + A_1 \overline{S_0} \overline{S_1} S_2 + A_2 \overline{S_0} S_1 \overline{S_2} + A_3 \overline{S_0} S_1 S_2 + A_4 S_0 \overline{S_1} \overline{S_2} + A_5 S_0 \overline{S_1} S_2 + A_6 S_0 S_1 \overline{S_2} + A_7 S_0 S_1 S_2$$

Figure: 2 Equation_LUT_8Bit

2.- FUNCTIONAL DESCRIPTION

In this lab, a "megafunction" was used, which is located in the "gates" - "mux" folder. In this case, it allows the use of a LUT (Look-Up Table), which is adaptive — meaning that based on the parameters set for the inputs, it will dynamically modify the selector part.

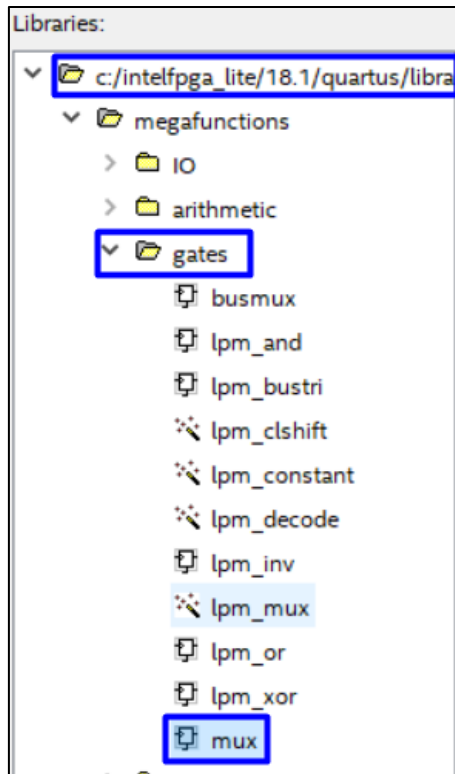
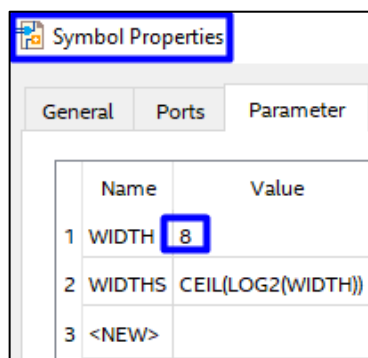


Figure: 3 Megafunction_Mux

The parameter to be changed is in the function's properties.



Parameter	Value
WIDTH	8
WIDTHS	CEIL(LOG2(WIDTH))

Figure: 4 Mux_Properties

The consideration for this implementation consisted of the 8 input bits in its definition (1 byte), and additionally, the change to 3 selectors.

The circuit was defined as follows:

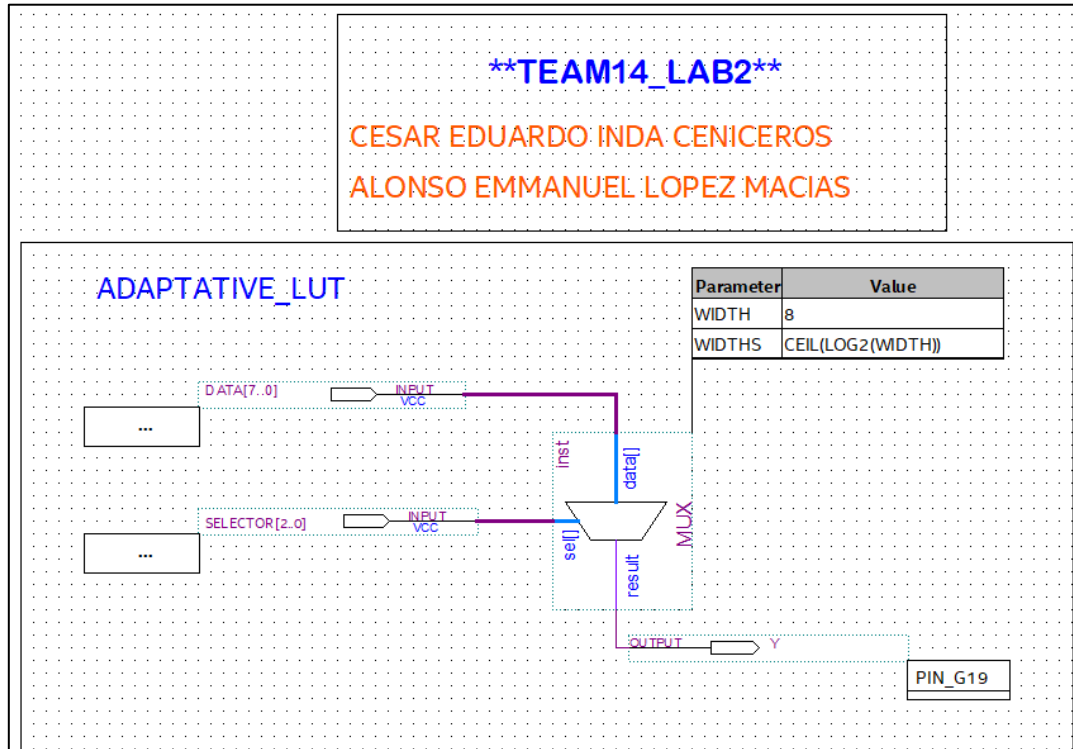


Figure: 5 AdaptativeLUT_Circuit

RTL VIEWER

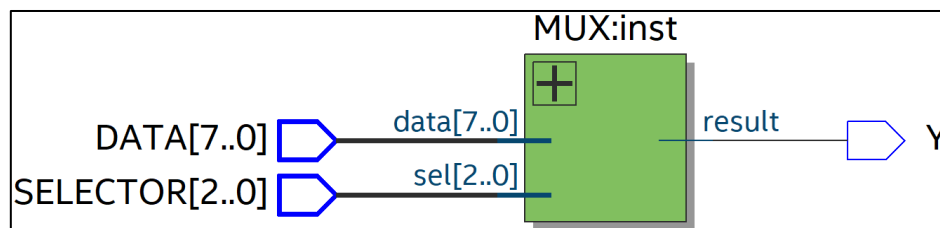


Figure: 6 RTL_AdaptativeLUT

The pin assignment in the “Pin Planner” was as follows:

Node Name	Direction	Location
DATA[7]	Input	PIN_AB26
DATA[6]	Input	PIN_AD26
DATA[5]	Input	PIN_AC26
DATA[4]	Input	PIN_AB27
DATA[3]	Input	PIN_AD27
DATA[2]	Input	PIN_AC27
DATA[1]	Input	PIN_AC28
DATA[0]	Input	PIN_AB28
SELECTOR[2]	Input	PIN_Y23
SELECTOR[1]	Input	PIN_Y24
SELECTOR[0]	Input	PIN_AA22
Y	Output	PIN_G19

Figure: 7 Pin_Planner

2.1.- True Tables

A LUT is a small memory that acts as a logic function. In this case, it consists of:

- 8 Inputs $[D0..D7]$
- 3 Selectors $[S0, S1, S2]$
- 1 Output $[Y]$

The selectors S2, S1, and S0 form a 3-bit binary number (from 000 to 111 = 0 to 7), and they select one of the 8 inputs. In this case, at a complete truth table, we would have:

- 8 data bits (input) = 256 combinations
- 3 selector bits = 8 combinations

So that $256 \times 8 = 2048$ rows of values. The next are the true table for the combinations by the selectors.

LUT 1				LUT 2			
S2	S1	S0	Y	S2	S1	S0	Y
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	1
0	1	0	0	0	1	0	1
0	1	1	1	0	1	1	0
1	0	0	0	1	0	0	1
1	0	1	0	1	0	1	0
1	1	0	1	1	1	0	0
1	1	1	1	1	1	1	1

Figure: 8 TrueTables_LUT

2.2.- Results

It was necessary to input the combination corresponding to what is set in Y for each table.

Combination1	Combination2
11001000	10010110

Figure: 9 TestCombinations



Figure: 10 Test1

LUT1 = 11001000

S = 011

LED0 = 1

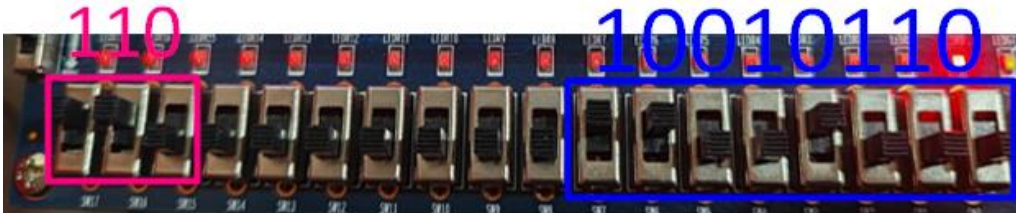


Figure: 11 Test2

LUT1 = 11001000

S = 110

LED0 = 1

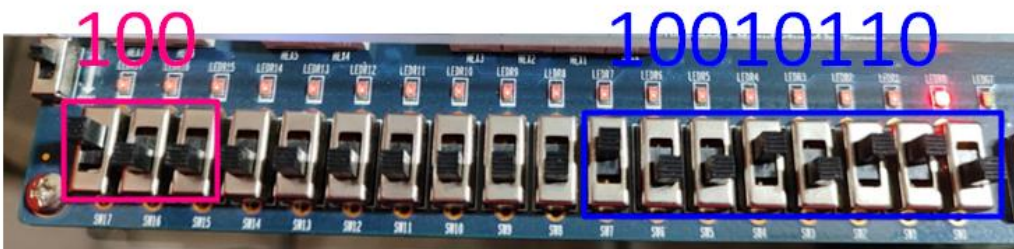


Figure: 12 Test3

LUT2 = 10010110

S = 100

LED0 = 1

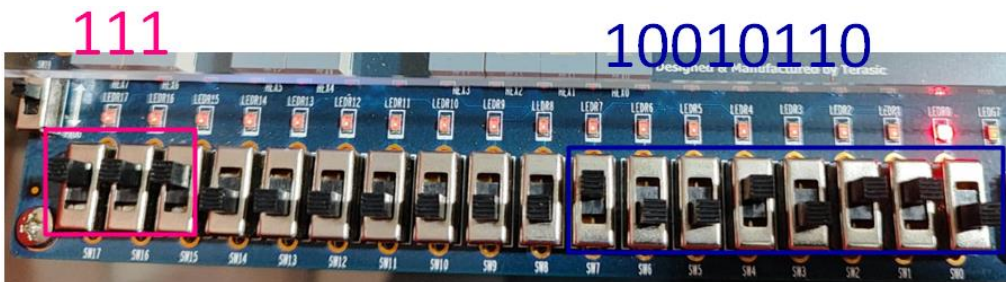


Figure: 13 Test4

LUT2 = 10010110

S = 111

LED0 = 1

In this case, a few scenarios are shown where the signal is verified, making it clear that if the test is performed with the other selector combinations, the result will also be correct.

2.3.- GitHub Repository

A branch was created in the course repository corresponding to Lab3_AdaptiveLUT.

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA/tree/QuartusLab3_AdaptiveLUT

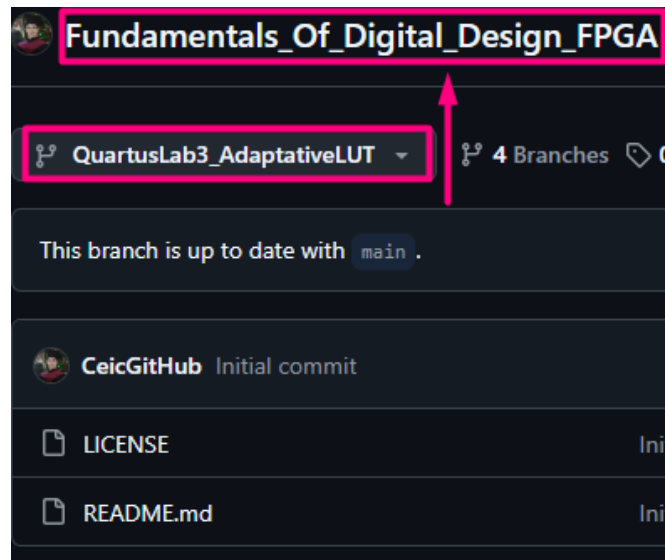


Figure: 14 GitHub Repository

3.- CHALLENGUES/ISSUES

Alonso Emmanuel López Macías

Some of the difficulties I encountered during the activity involved understanding what an adaptive LUT is and what its functionality was. Previously, I had a good understanding of logic gates and digital systems and many of their functions, but in this case, I had to grasp what the LUT actually did and also understand the activity itself taking into account that, to make changes, the parameter is automatically set by entering a value, and therefore the selector values are also updated accordingly.

Cesar Eduardo Inda Cenicerros

One complication I had was understanding that the functionality of the adaptive LUT in the Quartus interface automatically generates the selector parameters when adjusting the input buffer parameter. As of now, I haven't found an official Quartus guide that explains in more detail what each parameter of the block means, for example, pipelines or what that parameter is used for. I believe the most challenging part in this case was understanding what the block actually does, since there are several buxmux or similar elements that have different functional variables and blocks with other parameters.

4.- CONCLUSIONS

Alonso Emmanuel López Macías

Understanding what an adaptive LUT (Look-Up Table) is, is essential in digital system design because it allows the implementation of complex logic functions in an efficient, flexible, and configurable way. LUTs are fundamental building blocks in devices such as FPGAs, and their adaptive version offers the ability to dynamically modify their behavior based on specific parameters, such as the number of inputs or selectors. This adaptability facilitates the design and testing of digital circuits without the need to redesign logic structures from scratch, saving time and resources. Additionally, understanding how they work is key to optimizing performance, minimizing hardware usage, and comprehending how logic functions are implemented in modern development platforms such as Quartus.

Cesar Eduardo Inda Cenicerros

Adaptive LUTs (Look-Up Tables) play a crucial role in modern digital system design, especially within FPGA architecture. These programmable logic elements allow designers to implement a wide variety of logic functions by simply configuring memory-based truth tables, enabling rapid prototyping and flexible hardware development. The adaptive nature of these LUTs means they can adjust their internal configuration such as the number of inputs or operational mode based on design needs, significantly enhancing the scalability and reusability of logic circuits.

In the context of FPGAs, understanding how adaptive LUTs work is essential, as they are the foundation of logic implementation in these devices. Efficient use of LUTs directly impacts resource utilization, power consumption, and timing performance of the entire system. Therefore, mastering the concept and behavior of adaptive LUTs is not only a technical skill but a strategic advantage for engineers aiming to develop optimized, high-performance digital designs using FPGA technology.